

EPIC 06: DATABASE & DISTRIBUTED DATA DESIGN

Task 01: Database Design for Microservices

The task asks you to design:

1. **Database ownership**
2. **Indexes**
3. **Relationships**
4. **Query optimization**

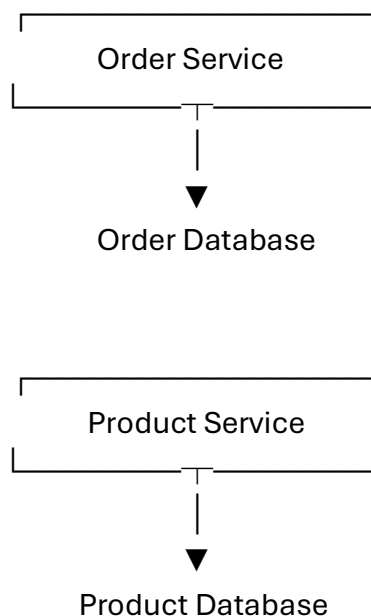
We can use the same microservices project: **Order Service, Product Service, and Notification Service.**

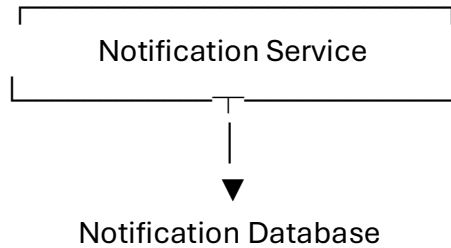
Step 1 — Database Ownership

In microservices, **each service should own its database.**

Do **not** allow Order Service to directly access Product Service's database.

Architecture





Ownership:

Service	Database	Main tables
Order Service	order_db	orders, order_items
Product Service	product_db	products
Notification Service	notification_db	notifications

Step 2 — Create Product Database

Product Service owns the products table.

```
CREATE DATABASE product_db;
```

Table:

```
CREATE TABLE products (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  price DECIMAL(10,2) NOT NULL,  
  stock INT NOT NULL  
);
```

Product index

Add an index for product name:

```
CREATE INDEX idx_product_name  
ON products(name);
```

Step 3 — Create Order Database

Order Service owns the order data.

```
CREATE DATABASE order_db;
```

Create orders:

```
CREATE TABLE orders (  
  
    id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  
    customer_id BIGINT NOT NULL,  
  
    status VARCHAR(30) NOT NULL,  
  
    created_at TIMESTAMP NOT NULL  
  
);
```

Create order_items:

```
CREATE TABLE order_items (  
  
    id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  
    order_id BIGINT NOT NULL,  
  
    product_id BIGINT NOT NULL,  
  
    quantity INT NOT NULL,
```

price DECIMAL(10,2) NOT NULL

);

Step 4 — Relationships

Inside the **Order Service database**, an order can have multiple order items.

Order

|

| 1

|

|

| *



Order Item

So:

orders.id



order_items.order_id

Create the relationship:

ALTER TABLE order_items

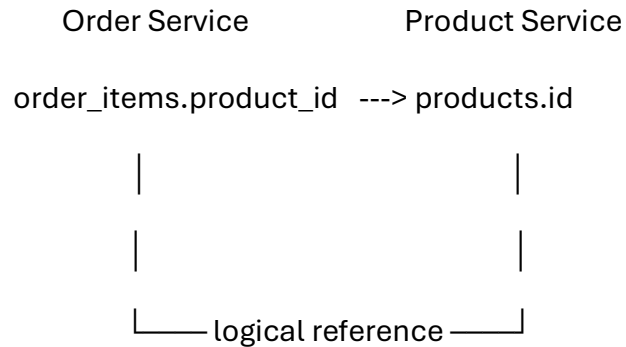
ADD CONSTRAINT fk_order_items_order

FOREIGN KEY (order_id)

REFERENCES orders(id);

Product relationship

product_id in order_items represents the Product Service's product ID.



There should **not** be a cross-database foreign key.

Step 5 — Add Indexes

Indexes improve the performance of frequently used queries.

For orders:

```
CREATE INDEX idx_orders_customer  
ON orders(customer_id);
```

For order status:

```
CREATE INDEX idx_orders_status  
ON orders(status);
```

For order items:

```
CREATE INDEX idx_order_items_order  
ON order_items(order_id);
```

For product ID:

```
CREATE INDEX idx_order_items_product  
ON order_items(product_id);
```

So, the important indexes are:

Table	Column	Index
orders	customer_id	idx_orders_customer
orders	status	idx_orders_status
order_items	order_id	idx_order_items_order
order_items	product_id	idx_order_items_product
products	name	idx_product_name

Step 6 — Query Optimization

Bad approach

Avoid retrieving unnecessary columns:

```
SELECT *  
  
FROM orders  
  
WHERE customer_id = 101;
```

Better approach

Retrieve only the required columns:

```
SELECT id, status, created_at  
FROM orders  
WHERE customer_id = 101;
```

The index on customer_id helps this query.

Step 7 — Optimize Order Item Query

Instead of retrieving unnecessary data:

```
SELECT *  
  
FROM order_items
```

```
WHERE order_id = 1001;
```

Use:

```
SELECT product_id, quantity, price  
FROM order_items  
WHERE order_id = 1001;
```

The `idx_order_items_order` index helps locate the items quickly.

Step 8 — Use Pagination

If a customer has thousands of orders, don't retrieve everything at once.

Use pagination:

```
SELECT id, status, created_at  
FROM orders  
WHERE customer_id = 101  
ORDER BY created_at DESC  
LIMIT 10 OFFSET 0;
```

For the next page:

```
LIMIT 10 OFFSET 10;
```

This reduces the amount of data returned to the application.

Step 9 — Check Query Performance

Use `EXPLAIN` to check whether the database is using an index.

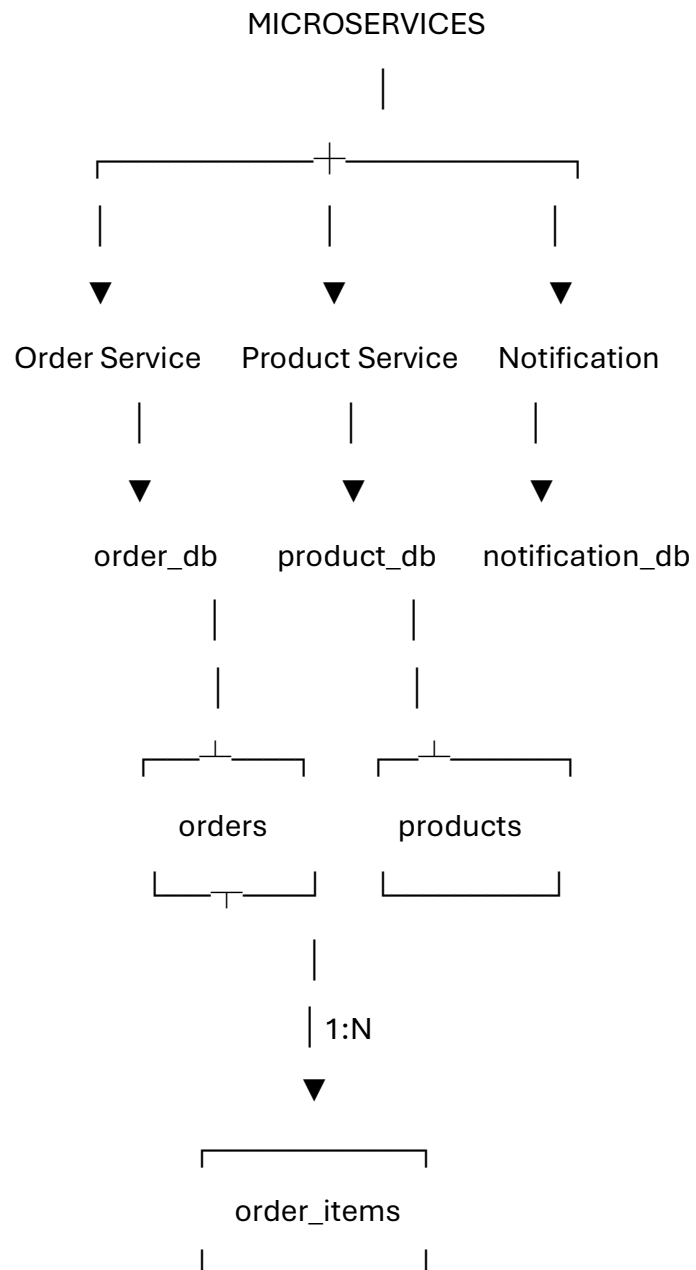
Example:

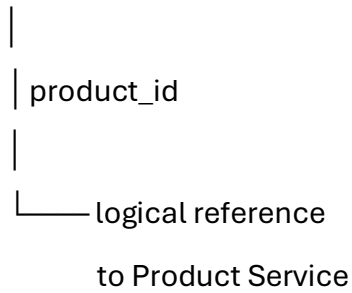
```
EXPLAIN  
SELECT id, status, created_at
```

FROM orders
WHERE customer_id = 101;

Check the execution plan for index usage.

Step 10 — Final Database Design





Conclusion:

Requirement	Implementation
Database ownership	One database per service
Indexes	Customer, status, order, product, name
Relationships	orders → order_items
Cross-service relationship	Logical productId, not DB foreign key
Query optimization	Select required columns
Pagination	LIMIT/OFFSET
Performance analysis	EXPLAIN

Task 01 completed: **Database Design for Microservices.**

